

Les Listes

Définition d'une liste

C'est une collection ordonnée et modifiable d'éléments éventuellement hétérogènes.

```
>>> nombres = [5, 38, 10, 25]
```

```
>>> mots = ["fromage", "confiture", "chocolat"]
```

```
>>> L = [5000, "Big Data", 3.1416, ["Bon", "Jour", 1947]]
```

Dans le dernier exemple ci-dessus, nous avons rassemblé un entier, une chaîne, un réel et même une liste, pour vous rappeler que l'on peut combiner dans une liste des données de n'importe quel type, y compris des listes.

Accès aux éléments d'une liste

Pour accéder aux éléments d'une liste, on utilise les mêmes méthodes (index, découpage en tranches) que pour accéder aux caractères d'une chaîne :

```
>>> print(nombres[2])
```

```
10
```

```
>>> print(nombres[1:3])
```

```
[38, 10]
```

```
>>> print(nombres[2:3])
```

```
[10]
```

```
>>> print(nombres[2:])
```

```
[10, 25]
```

```
>>> print(nombres[:2])
```

```
[5, 38]
```

```
>>> print(nombres[-1])
```

```
25
```

```
>>> print(nombres[-2])
```

```
10
```

Les Listes

Les listes sont modifiables

Les listes sont des *séquences modifiables*.

Exemples :

```
>>> nombres[0] = 17
>>> print(nombres)
[17, 38, 10, 25]
```

Les listes sont des objets

Sous Python, les listes sont des objets pour les quels on peut appliquer des **méthodes** (fonctions) particulièrement efficaces. En voici quelques-unes :

```
>>> nombres = [17, 38, 10, 25, 72]
>>> nombres.sort()           # trier la liste
>>> nombres
[10, 17, 25, 38, 72]
>>> nombres.append(12)       # ajouter un élément à la fin
>>> nombres
[10, 17, 25, 38, 72, 12]
>>> nombres.reverse()       # inverser l'ordre des éléments
>>> nombres
[12, 72, 38, 25, 17, 10]
```

Les Listes

```
>>> nombres.index(17)          # retrouver l'index d'un élément
```

```
4
```

```
>>> nombres.remove(38)        # enlever (effacer) un élément
```

```
>>> nombres
```

```
[12, 72, 25, 17, 10]
```

En plus de ces méthodes, vous disposez encore de l'instruction intégrée **del**, qui vous permet d'effacer un ou plusieurs éléments à partir de leur(s) index :

```
>>> del(nombres[2])
```

```
>>> nombre
```

```
s[12, 72, 17, 10]
```

```
>>> del(nombres[1:3])
```

```
>>> nombres
```

```
[12, 10]
```

Insertion d'un ou plusieurs éléments n'importe où dans une liste

```
>>> mots = ['jambon', 'fromage', 'confiture', 'chocolat']
```

```
>>> mots[2:2] = ["miel"]          # équivalent à mots=mots[:2] + ["miel"] + mots[2:]
```

```
>>> mots
```

```
['jambon', 'fromage', 'miel', 'confiture', 'chocolat']
```

```
>>> mots[5:5] = ['saucisson', 'ketchup']
```

```
>>> mots
```

```
['jambon', 'fromage', 'miel', 'confiture', 'chocolat', 'saucisson', 'ketchup']
```

Les Listes

Suppression/remplacement d'éléments

```
>>> mots[2:5] = []          # [] désigne une liste vide
>>> mots
['jambon', 'fromage', 'saucisson', 'ketchup']
>>> mots[1:3] = ['salade']
>>> mots['jambon', 'salade', 'ketchup']
>>> mots[1:] = ['mayonnaise', 'poulet', 'tomate']
>>> mots['jambon', 'mayonnaise', 'poulet', 'tomate']
```

Création d'une liste de nombres à l'aide de la fonction range()

```
>>> L=list(range(10))
>>> L
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> list(range(5,13))
[5, 6, 7, 8, 9, 10, 11, 12]
>>> list(range(3,16,3))
[3, 6, 9, 12, 15]
```

Les arguments négatifs sont autorisés :

```
>>> list(range(10, -10, -3))
[10, 7, 4, 1, -2, -5, -8]
```

Les Listes

Parcours d'une liste à l'aide de **for**, **range()** et **len()**

L'instruction **for** est l'instruction idéale pour parcourir une liste :

```
>>> prov = ['La','raison','du','plus','fort','est','toujours','la','meilleure']
>>> for mot in prov:
...     print(mot, end = ' ')
```

La raison du plus fort est toujours la meilleure

Si vous voulez parcourir une gamme d'entiers, la fonction **range()** s'impose :

```
>>> for n in range(10, 18, 3):
...     print(n, n**2, n**3)...
```

10 100 1000

13 169 2197

16 256 4096

Une liste en mode compréhension :

```
>>> S=[i**2 for i in range(4)]
```

```
>>> S
```

```
[0, 1, 4, 9]
```

```
>>> L2=[x for x in L1 if x%3==0]
```

```
>>> L2
```

```
[0, 3, 6, 9]
```

Les Listes

Opérations sur les listes

On peut appliquer aux listes les opérateurs **+** (concaténation) et ***** (multiplication) :

```
>>> fruits = ['orange','citron']
```

```
>>> legumes = ['poireau','oignon','tomate']
```

```
>>> fruits + legumes
```

```
['orange', 'citron', 'poireau', 'oignon', 'tomate']
```

```
>>> fruits * 3
```

```
['orange', 'citron', 'orange', 'citron', 'orange', 'citron']
```

L'opérateur ***** est particulièrement utile pour créer une liste de **n** éléments identiques :

```
>>> T = [0]*7
```

```
>>> T
```

```
[0, 0, 0, 0, 0, 0, 0]
```

Supposons par exemple que vous voulez créer une liste **B** qui contient le même nombre d'éléments qu'une autre liste **A**. Vous pouvez obtenir ce résultat de différentes manières, l'une des plus simples consiste à effectuer : **B = [0]*len(A)**.

```
>>>'orange' in fruits
```

```
True
```

```
>>>len(fruits)
```

```
2
```

Les Listes

`L.index(x,a,b)` retourne la position de la première occurrence de `x` dans la liste `L` entre les deux positions `a` et `b`. si `b` n'est pas donné la recherche s'effectue de la position `a` à la fin.

`L.index(x)` retourne la position de la première occurrence de `x` dans la liste `L` (de début à la fin).

```
>>> T=[6,4,8,4,9,4,7,4,9]
```

```
>>> m=T.index(4,2,7)
```

```
>>> m
```

```
3
```

```
>>> m=T.index(4,6)
```

```
>>> m
```

```
7
```

```
>>> m=T.index(4,8)
```

```
Traceback (most recent call last):
```

```
File "<console>", line 1, in <module>
```

```
ValueError: 4 is not in list
```

```
>>> m=T.index(9,8)
```

```
>>> m
```

```
8
```

Les Listes

```
>>> T=[6,4,8,4,9,4,7,4,9]
>>> T.remove(T[3])
>>> T
[6, 8, 4, 9, 4, 7, 4, 9]
>>> del(T[2])
>>> T
[6, 8, 9, 4, 7, 4, 9]
>>> a=T.pop()
>>> a
9
>>> T
[6, 8, 9, 4, 7, 4]
>>> b=T.pop(-2)
>>> T
[6, 8, 9, 4, 4]
>>> b=T.pop(1)
>>> b
8
```


Les Listes

```
>>> T=[6,4,8,4,9,4,7,4,9]
>>> T.remove(T[3])
>>> T
[6, 8, 4, 9, 4, 7, 4, 9]
>>> del(T[2])
>>> T
[6, 8, 9, 4, 7, 4, 9]
>>> a=T.pop()
>>> a
9
>>> T
[6, 8, 9, 4, 7, 4]
>>> b=T.pop(-2)
>>> T
[6, 8, 9, 4, 4]
>>> b=T.pop(1)
>>> b
8
```

Exercice:

Quel est le résultat du programme suivant:

```
L=[10,8,0,6,7,0,5,9,0,6,3]
for i in range(len(L)):
    if (L[i]==0):
        del(L[i])
print(L)
```

Les Listes

```
>>> T=[6,4,8,4,9,4,7,4,9]
>>> T.append([1,2,3])
>>> T
[6, 4, 8, 4, 9, 4, 7, 4, 9, [1, 2, 3]]
>>> T.extend([7,8,9])
>>> T
[6, 4, 8, 4, 9, 4, 7, 4, 9, [1, 2, 3], 7, 8, 9]
>>> T.insert(2,10000) # insère 10000 dans la position 2 de
                        la liste T
>>> T
[6, 4, 10000, 8, 4, 9, 4, 7, 4, 9, [1, 2, 3], 7, 8, 9]

>>> L=["Bon","Jour","Monsieur"]
>>> C="***".join(L)
>>> C
'Bon***Jour***Monsieur'
```

Les Listes

```
liste = [(1, 2), (3, 4), (5, 6)]
```

```
# 1. Extraire le premier élément de chaque tuple
```

```
premiers = [t[0] for t in liste]
```

```
# 2. Calculer la somme de chaque tuple
```

```
somme_tuples = [a + b for a, b in liste]
```

```
# 3. Filtrer les tuples dont la somme est > 5
```

```
filtres = [(a, b) for a, b in liste if a + b > 5]
```

```
# 4. Transformer chaque tuple (multiplier chaque élément par 10)
```

```
transformes = [(a*10, b*10) for a, b in liste]
```

```
print("Premiers :", premiers)
```

```
print("Sommes :", somme_tuples)
```

```
print("Filtres :", filtres)
```

```
print("Transformés :", transformes)
```